User Process Execution

Lifetime of Process

Contents

- System calls related to process execution
  - `fork(void)`
  - `exec(const char *, char **)`
  - `wait(int *)`
  - `exit(int)`
  - `kill(int)`

fork

- Creates a new process by duplicating the calling process.
- The child process and the parent process run in separate memory space.
- Both processes continue to execute from the point where the `fork()` system call returns.
- The parent receives the child's PID as the return value, while the child receives 0.

fork

1. Allocates new process control block using `allocproc()`.
2. Copies user memory from parent to new process page table.

```
$ vim kernel/proc.c

259  int
260  kfork(void)
261  {
262    int i, pid;
263    struct proc *np;
264    struct proc *p = myproc();
265
266    // Allocate process.
267    if((np = allocproc()) == 0){
268        return -1;
269    }
270
271    // Copy user memory from parent to child.
272    if(uvmcopy(p->pagetable, np->pagetable, p->sz < 0){
273      freeproc(np);
274      release(&np->lock);
275      return -1;
276    }
277    np->sz = p->sz;
278
...
```

allocproc

- `allocproc()` allocates and initializes a new process control block(PCB) to prepare the process for execution.
- It searches the process table for a process in the `UNUSED` state, then allocates the necessary resources and initializes the process.

allocproc

1. Searches the process table for an UNUSED process
2. If found, jump to the found label.
3. If the table is full, return 0.

```
$ vim kernel/proc.c

109  static struct proc*
110  allocproc(void)
111  {
112    struct proc *p;
113
114    for(p = proc; p < &proc[NPROC]; p++) {
115      acquire(&p->lock);
116      if(p->state == UNUSED) {
117        goto found;
118       } else {
119        release(&p->lock);
120       }
121    }
122    return 0;
123
124  found:
125    p->pid = allocpid();
126    p->state = USED;
127
128    // Allocate a trapframe page
...
```

found label in allocproc

1. Allocates a new PID.
2. Changes its state to USED.
3. Allocates a trapframe page.
4. Allocates an empty user page table.
5. Initializes the process context and set the return address to `forkret()`.

```
$ vim kernel/proc.c

110  allocproc(void)
...
124  found:
125    p->pid = allocpid();
126    p->state = USED;
127
128    // Allocate a trapframe page
129    if((p->trapframe = (struct trapframe *)kalloc()) ...
...
135    // An empty user page table.
136    p->pagetable = proc_pagetable(p);
137    if(p->pagetable == 0){
...
143    // Set up new context to start executing at forkret,
144    // which returns to user space.
145    memset(&p->context, 0, sizeof(p->context));
146    p->context.ra = (uint64)forkret;
147    p->context.sp = p->kstack + PGSIZE;
148
149    return p;
150  }
```

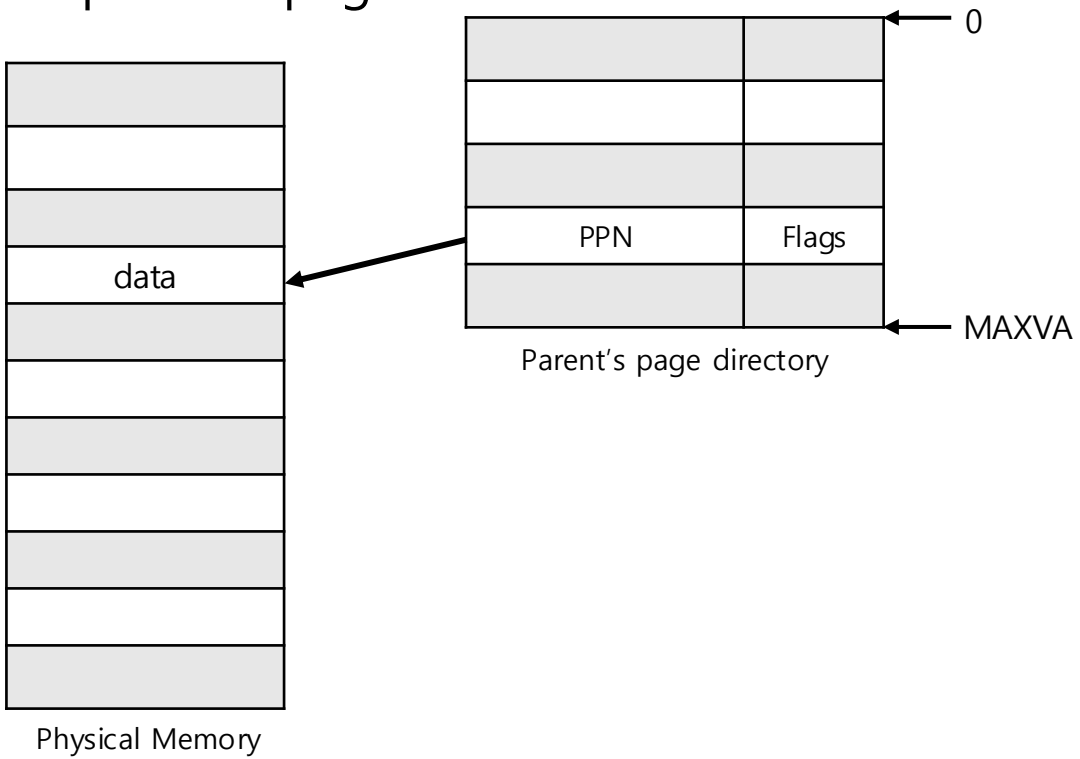
fork

1. Allocates new process control block using `allocproc()`.
2. Copies user memory from parent to new process page table.

```
$ vim kernel/proc.c

259  int
260  kfork(void)
261  {
262    int i, pid;
263    struct proc *np;
264    struct proc *p = myproc();
265
266    // Allocate process.
267    if((np = allocproc()) == 0){
268        return -1;
269    }
270
271    // Copy user memory from parent to child.
272    if(uvmcopy(p->pagetable, np->pagetable, p->sz < 0){
273      freeproc(np);
274      release(&np->lock);
275      return -1;
276    }
277    np->sz = p->sz;
278
...
```

fork

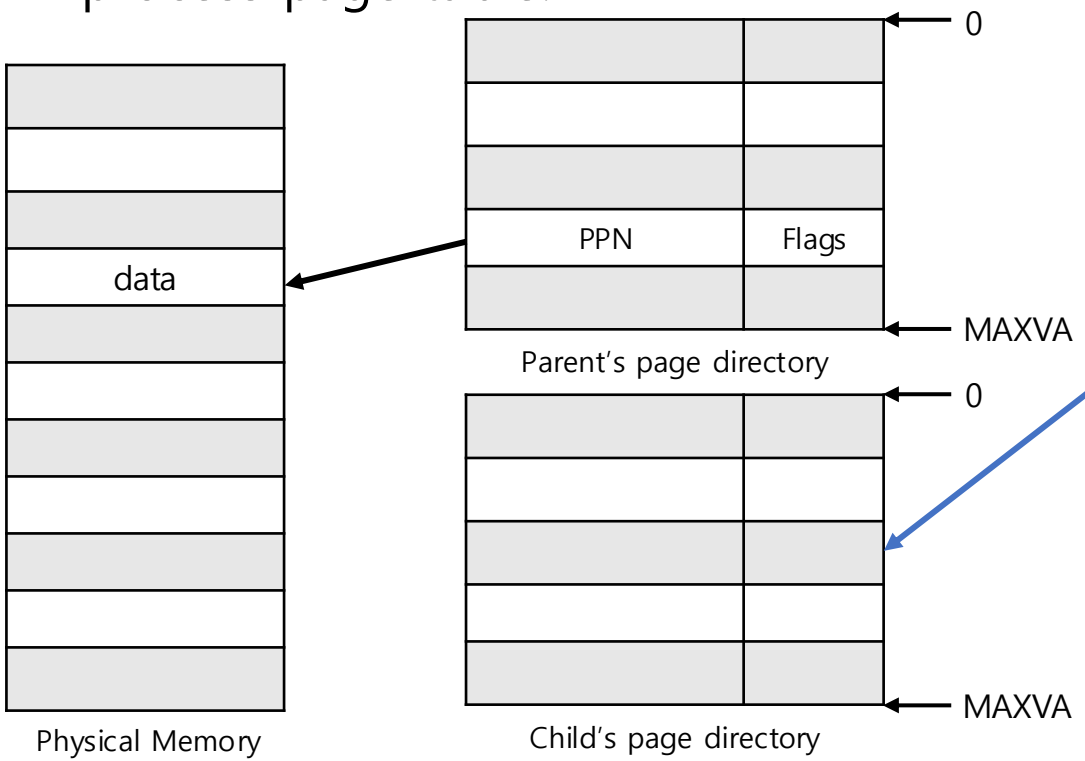
1. Allocates new process control block using `allocproc()`.
2. Copies user memory from parent to new process page table.




```
$ vim kernel/proc.c

259  int
260  kfork(void)
261  {
262    int i, pid;
263    struct proc *np;
264    struct proc *p = myproc();
265
266    // Allocate process.
267    if((np = allocproc()) == 0){
268        return -1;
269    }
270
271    // Copy user memory from parent to child.
272    if(uvmcopy(p->pagetable, np->pagetable, p->sz < 0){
273      freeproc(np);
274      release(&np->lock);
275      return -1;
276    }
277    np->sz = p->sz;
278
...
```

fork

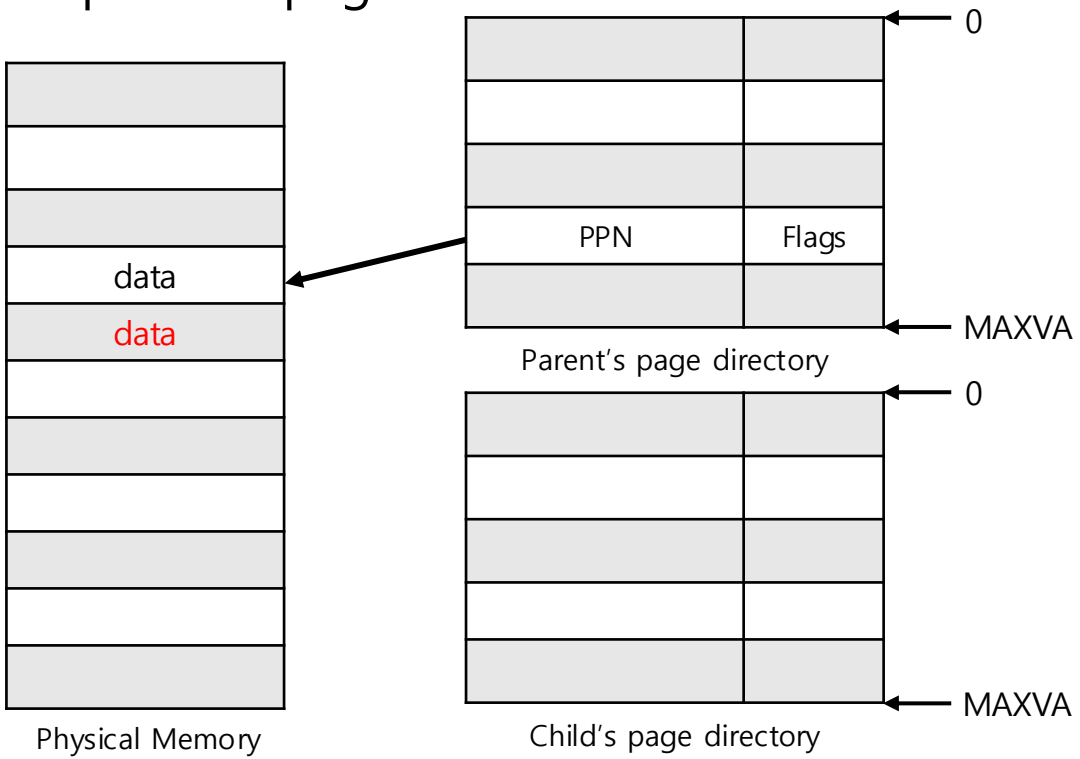
1. Allocates new process control block using `allocproc()`.
2. Copies user memory from parent to new process page table.




```
$ vim kernel/proc.c

259  int
260  kfork(void)
261  {
262    int i, pid;
263    struct proc *np;
264    struct proc *p = myproc();
265
266    // Allocate process.
267    if((np = allocproc()) == 0){
268      return -1;
269    }
270
271    // Copy user memory from parent to child.
272    if(uvmcopy(p->pagetable, np->pagetable, p->sz < 0){
273      freeproc(np);
274      release(&np->lock);
275      return -1;
276    }
277    np->sz = p->sz;
278
...
```

fork

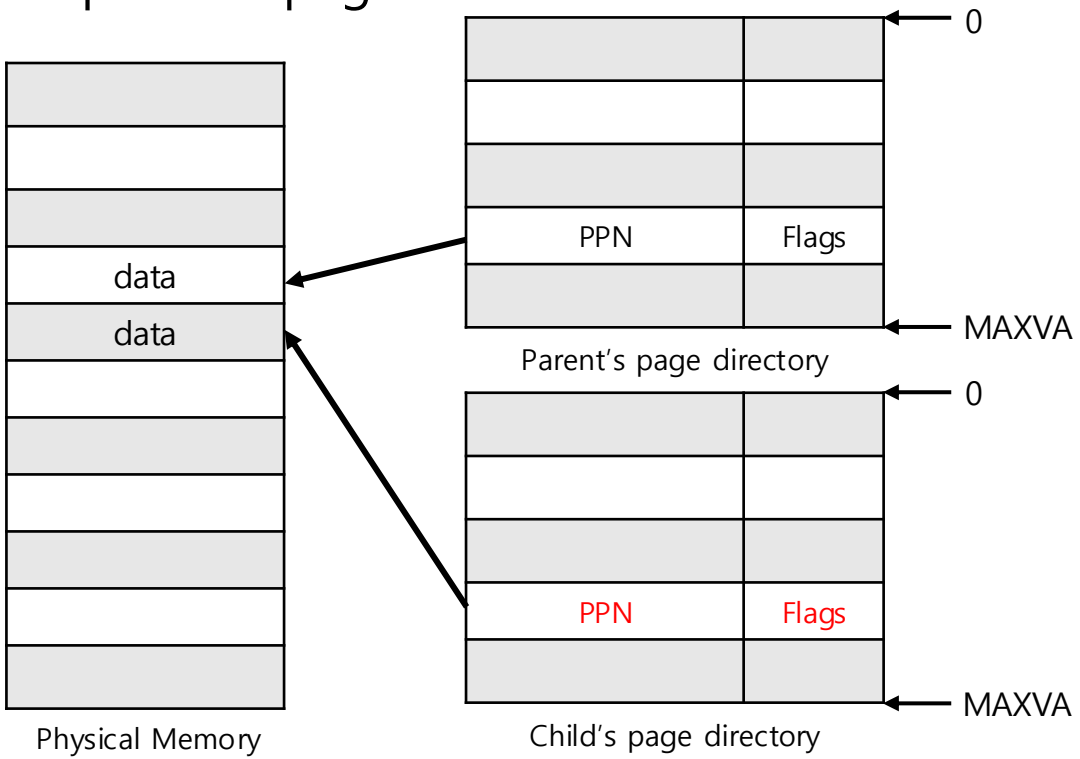
1. Allocates new process control block using `allocproc()`.
2. Copies user memory from parent to new process page table.




```
$ vim kernel/proc.c

259  int
260  kfork(void)
261  {
262    int i, pid;
263    struct proc *np;
264    struct proc *p = myproc();
265
266    // Allocate process.
267    if((np = allocproc()) == 0){
268        return -1;
269    }
270
271    // Copy user memory from parent to child.
272    if(uvmcopy(p->pagetable, np->pagetable, p->sz) < 0){
273      freeproc(np);
274      release(&np->lock);
275      return -1;
276    }
277    np->sz = p->sz;
278
...
```

fork

1. Allocates new process control block using `allocproc()`.
2. Copies user memory from parent to new process page table.




```
$ vim kernel/proc.c

259  int
260  kfork(void)
261  {
262    int i, pid;
263    struct proc *np;
264    struct proc *p = myproc();
265
266    // Allocate process.
267    if((np = allocproc()) == 0){
268        return -1;
269    }
270
271    // Copy user memory from parent to child.
272    if(uvmcopy(p->pagetable, np->pagetable, p->sz) < 0){
273      freeproc(np);
274      release(&np->lock);
275      return -1;
276    }
277    np->sz = p->sz;
278
...
```

fork

3. Copies parent's **TRAPFRAME**.
4. Sets a0 register of the child's **TRAPFRAME** to 0.
5. Copies parent's file descriptors and current working directory.
6. Copies parent's process name.

```
$ vim kernel/proc.c

260  kfork(void)
...
279    // copy saved user registers.
280    *(np->trapframe) = *(p->trapframe);
281
282    // Cause fork to return 0 in the child.
283    np->trapframe->a0 = 0;
284
285    // increment reference counts on open ...
286    for(i = 0; i < NOFILE; i++)
287      if(p->ofile[i])
288        np->ofile[i] = filedup(p->ofile[i]);
289    np->cwd = idup(p->cwd);
290
291    safestrcpy(np->name, p->name, sizeof(p->name));
292
...
```

fork

7. Sets the parent of the newly created process to the on that called `fork()`.
8. Sets new process's state to `RUNNABLE`.
   - Now child process is ready to be scheduled
9. Parent process finishes the fork system call routine.

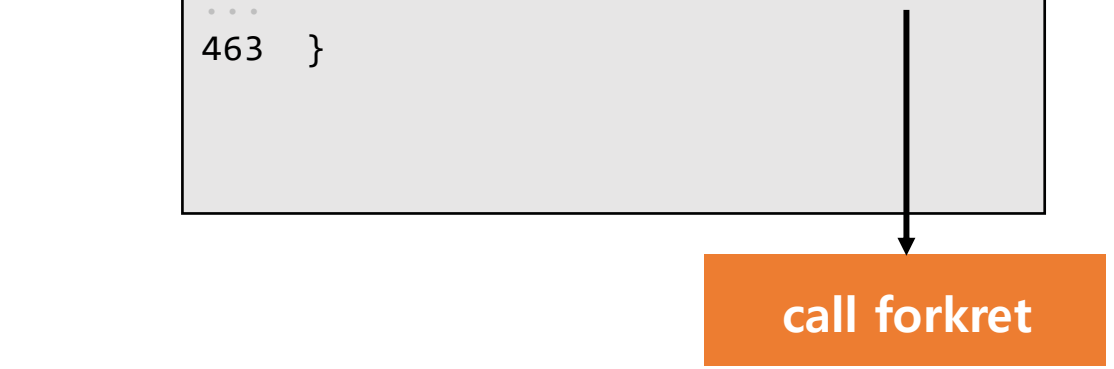
```
$ vim kernel/proc.c

260  kfork(void)
...
293    pid = np->pid;
294
295    release(&np->lock);
296
297    acquire(&wait_lock);
298    np->parent = p;
299    release(&wait_lock);
300
301    acquire(&np->lock);
302    np->state = RUNNABLE;
303    release(&np->lock);
304
305    return pid;
306  }
```

Where does the new process begin?

Where does the new process begin?

```
$ vim kernel/proc.c

109  static struct proc*
110  allocproc(void)
...
144    // which returns to user space.
145    memset(&p->context, 0, sizeof(p->context));
146    p->context.ra = (uint64)forkret;
147    p->context.sp = p->kstack + PGSIZE;
148
149    return p;
150  }
```

```
$ vim kernel/proc.c

424  void
425  scheduler(void)
426  {
...
447    p->state = RUNNING;
448    c->proc = p;
449    swtch(&c->context, &p->context);
...
463  }
```



forkret

```
$ vim kernel/proc.c

505  void
506  forkret(void)
507  {
508    extern char userret[];
509    static int first = 1;
510    struct proc *p = myproc();
511
512    // Still holding p->lock from scheduler.
513    release(&p->lock);
514
515    if(first) {
...
527      p->trapframe->a0 = kexec("/init", (char *[]){ "/init", 0 });
...
531    }
532
533    // return to user space, mimicing usertrap()'s return.
534    prepare_return();
535    uint64 satp = MAKE_SATP(p->pagetable);
536    uint64 trampoline_userret = TRAMPOLINE + (userret - trampoline);
537    ((void (*)(uint64))trampoline_userret)(satp);
538  }
```

1. Releases the process lock held by scheduler.
2. If it is the first process, executes the init program.
3. Executes `prepare_return()` and jump back to user space.

forkret

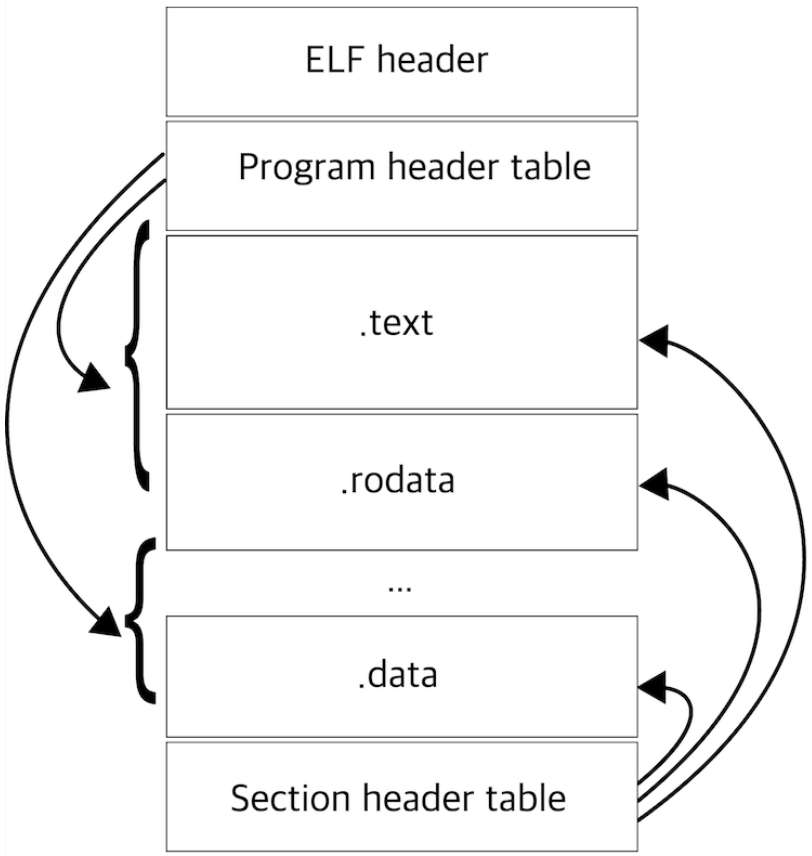
```
$ vim kernel/proc.c

505  void
506  forkret(void)
507  {
508    extern char userret[];
509    static int first = 1;
510    struct proc *p = myproc();
511
512    // Still holding p->lock from scheduler.
513    release(&p->lock);
514
515    if(first) {
...
527      p->trapframe->a0 = kexec("/init", (char *[]){ "/init", 0 });
...
531    }
532
533    // return to user space, mimicing usertrap()'s return.
534    prepare_return();
535    uint64 satp = MAKE_SATP(p->pagetable);
536    uint64 trampoline_userret = TRAMPOLINE + (userret - trampoline);
537    ((void (*)(uint64))trampoline_userret)(satp);
538  }
```

```
$ vim kernel/proc.c

494  yield(void)
...
496    struct proc *p = myproc();
497    acquire(&p->lock);
498    p->state = RUNNABLE;
499    sched();
500    release(&p->lock);
501  }

$ vim kernel/trap.c

 38  usertrap(void)
...
 84    if(which_dev == 2)
 85      yield();
 86
 87    prepare_return();
...
 90    uint64 satp = MAKE_SATP(p-> ...
...
 93    return satp;
 94  }
```

exec

- exec() replaces the current process's memory with a new executable, effectively transforming the process into a new program.
- exec() returns -1 only if an error has occurred.
  - free all resources

ELF (Executable and Linkable Format)

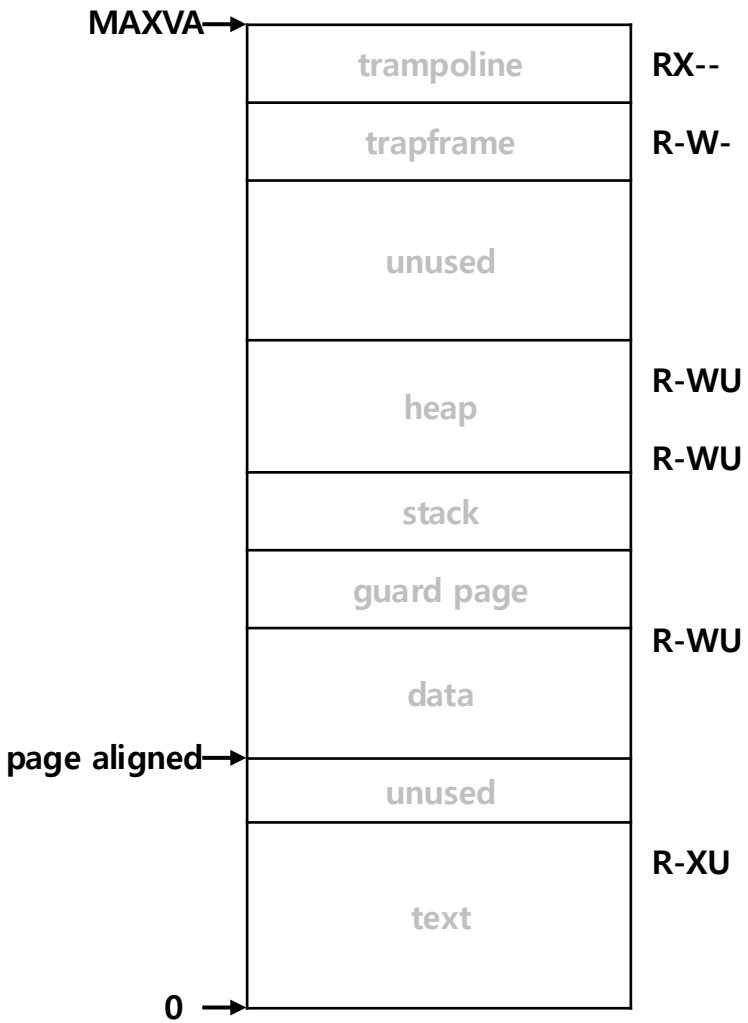


- ELF (Executable and Linkable Format) is a standard file format used in Unix-like systems to store executables, object code, shared libraries and core dumps.
- xv6 applications are described in the widely-used ELF format, defined in kernel/elf.h

exec

```
$ vim kernel/exec.c

 26   int
 27   kexec(char *path, char **argv)
 28   {
...
 41     if((ip = namei(path)) == 0){
...
 47     // Check ELF header
 48     if(readi(ip, 0, (uint64)&elf, 0, sizeof(elf) != ...
 49       goto bad;
...
 54
 55     if((pagetable = proc_pagetable(p)) == 0)
 56       goto bad;
...
```

1. Gets inode for a file to execute.
2. Reads the "ELF" header from the file in a disk.
3. Allocates a new page table and map trapframe & trampoline page into it.

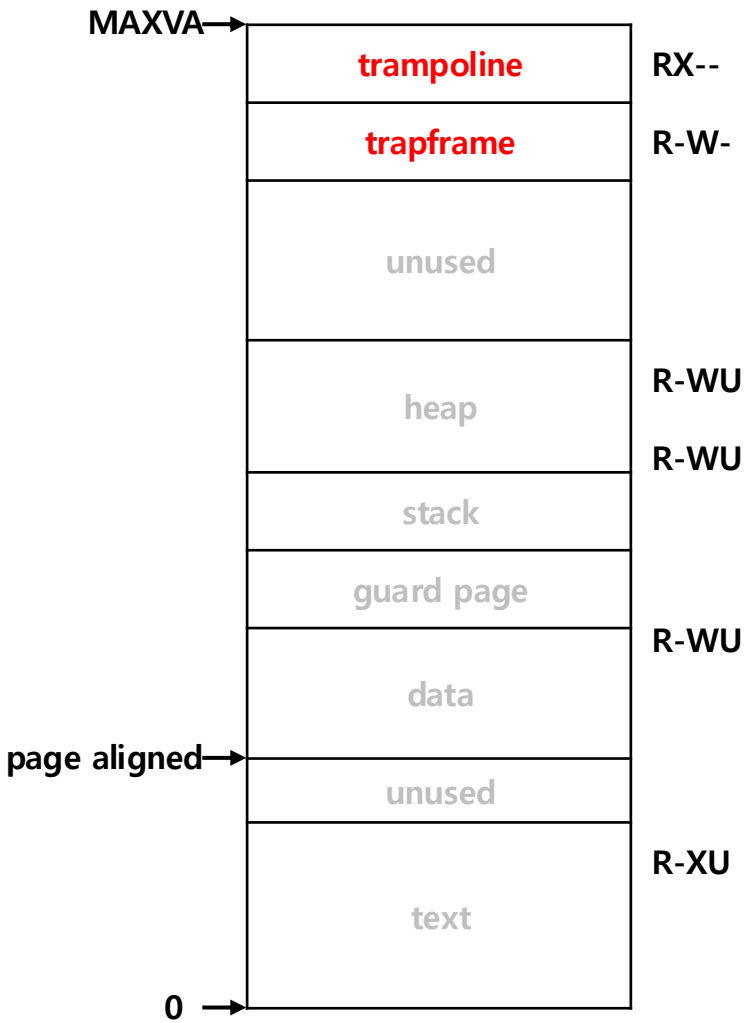


A process's user address space

exec

```
$ vim kernel/exec.c

 26   int
 27   kexec(char *path, char **argv)
 28   {
...
 41     if((ip = namei(path)) == 0){
...
 47     // Check ELF header
 48     if(readi(ip, 0, (uint64)&elf, 0, sizeof(elf) != ...
 49       goto bad;
...
 54
 55     if((pagetable = proc_pagetable(p)) == 0)
 56       goto bad;
...
```

1. Gets inode for a file to execute.
2. Reads the "ELF" header from the file in a disk.
3. Allocates a new page table and map trapframe & trampoline page into it.

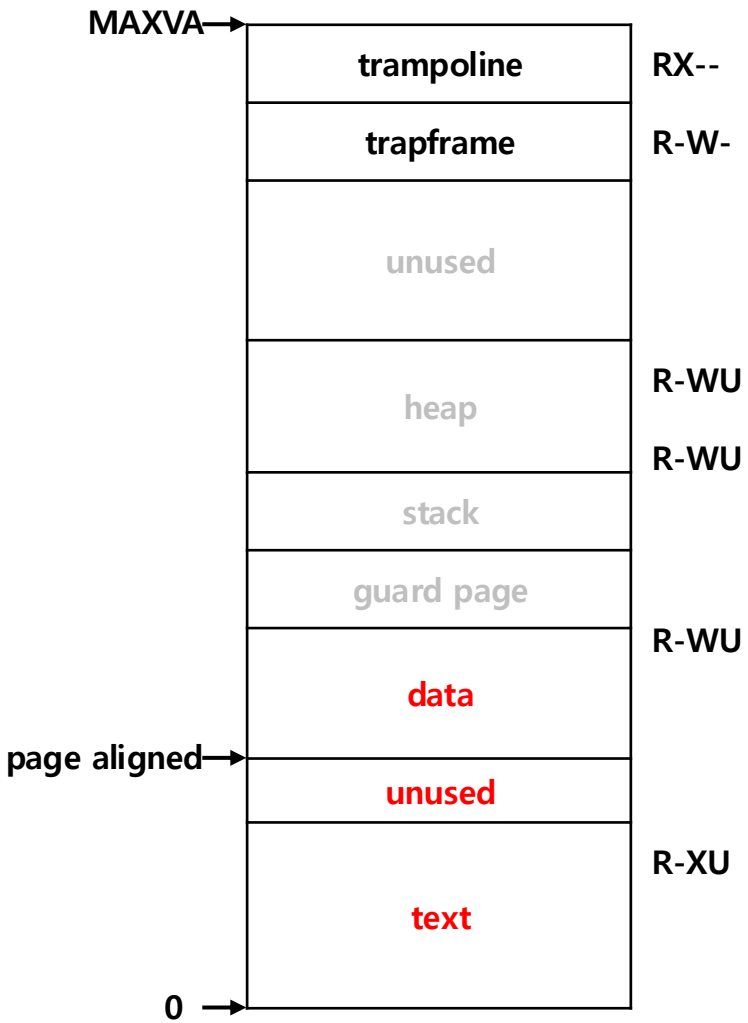


A process's user address space

exec

```
$ vim kernel/exec.c

 27   kexec(char *path, char **argv)
...
 58     // Load program into memory.
 59     for(i=0, off=elf.phoff; i<elf.phnum; i++, off+=sizeof(ph)){
 60       if(readi(ip, 0, (uint64)&ph, off, sizeof(ph)) != sizeof(ph))
 61         goto bad;
...
 71       if((sz1 = uvmalloc(pagetable, sz, ph.vaddr + ph.memsz ...
 72         goto bad;
 73       sz = sz1;
 74       if(loadseg(pagetable, ph.vaddr, ip, ph.off, ph.filesz) < 0)
 75         goto bad;
 76     }
...
```

4. Reads "program" headers from the ELF file.
5. Allocates memory and load each segment into the address space.

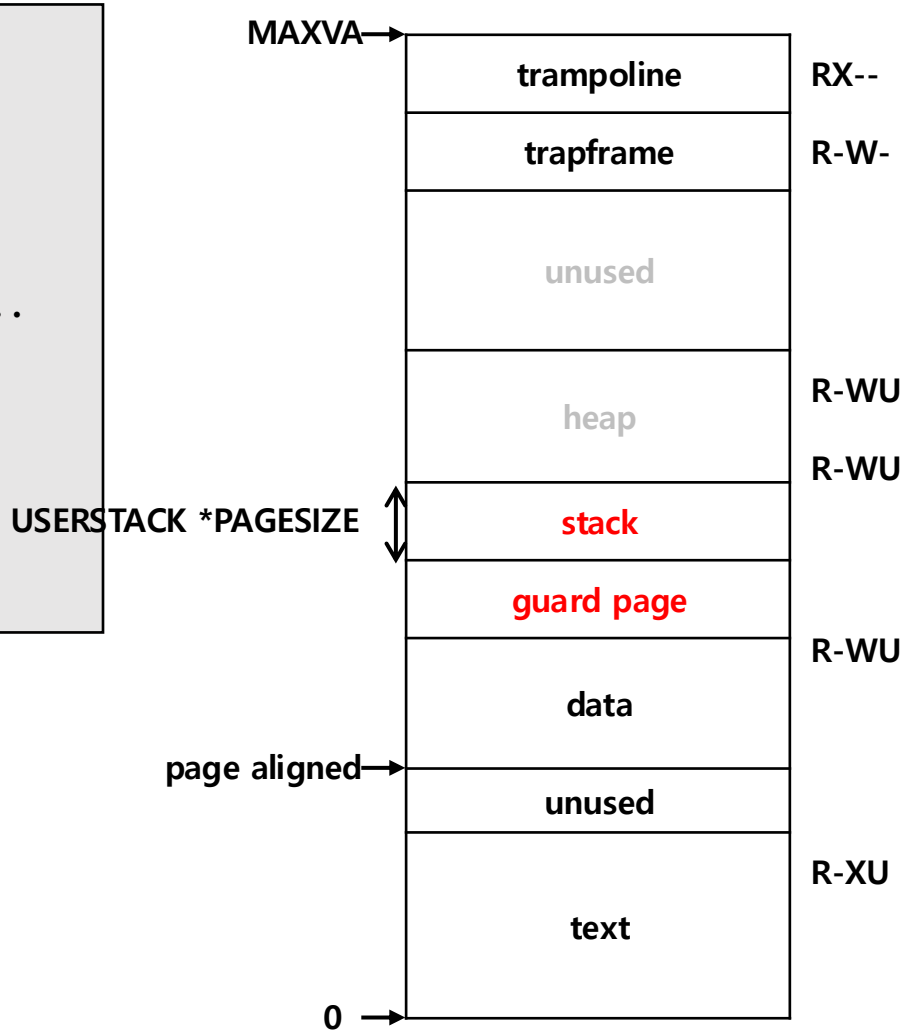


A process's user address space

exec

```
$ vim kernel/exec.c

 27   kexec(char *path, char **argv)
...
 87     sz = PGROUNDUP(sz);
 88     uint64 sz1;
 89     if((sz1 = uvmalloc(pagetable, sz, sz + (USERSTACK+1)*PGSIZE...
 90       goto bad;
 91     sz = sz1;
 92     uvmclear(pagetable, sz-(USERSTACK+1)*PGSIZE);
 93     sp = sz;
 94     stackbase = sp - USERSTACK*PGSIZE;
...
```

6. Allocates and clear memory for the user stack and a guard page.
7. Initializes the stack pointer(sp) and stack base.

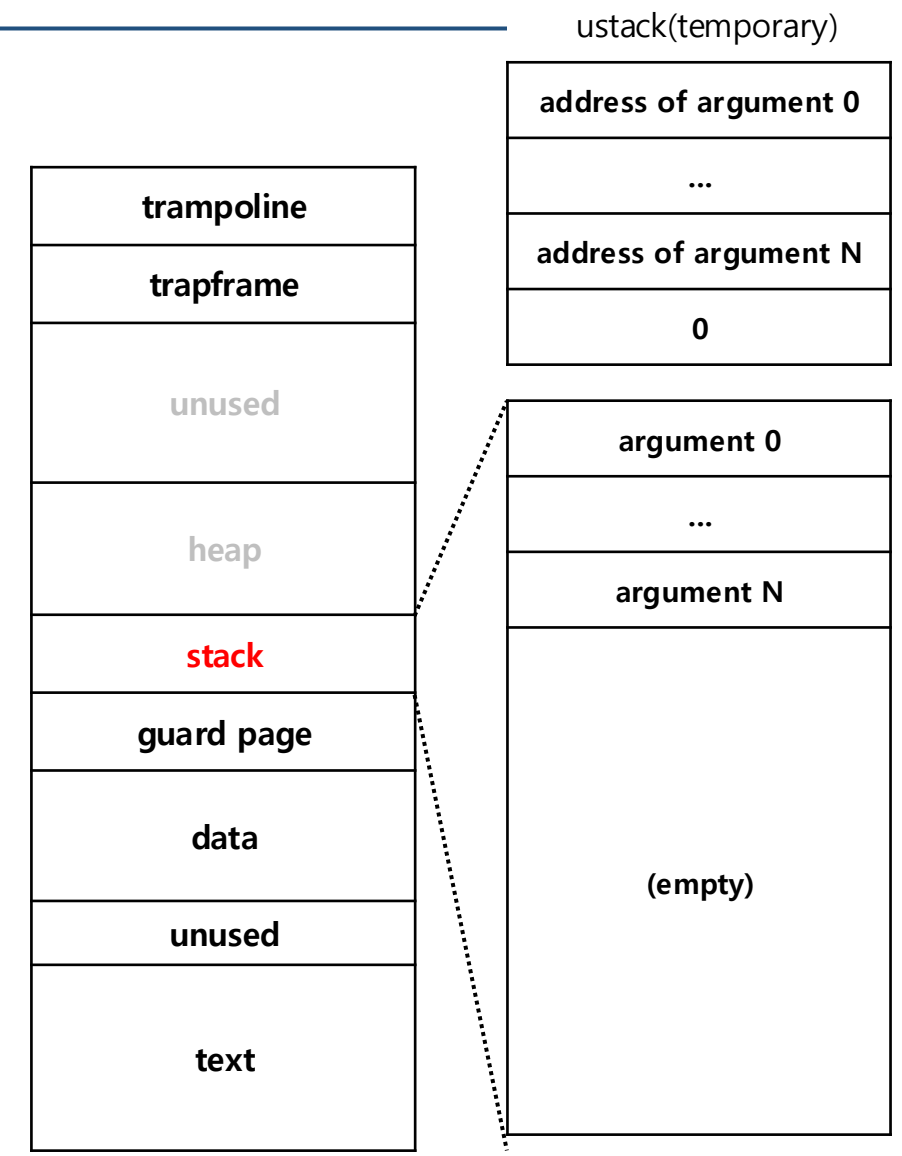


A process's user address space

exec

```
$ vim kernel/exec.c

 27   kexec(char *path, char **argv)
...
 98     for(argc = 0; argv[argc]; argc++) {
 99       if(argc >= MAXARG)
100         goto bad;
101       sp -= strlen(argv[argc]) + 1;
102       sp -= sp % 16; // riscv sp must be 16-byte aligned
...
105       if(copyout(pagetable, sp, argv[argc], strlen(argv[argc]) ...
106         goto bad;
107       ustack[argc] = sp;
108     }
109     ustack[argc] = 0;
...
```

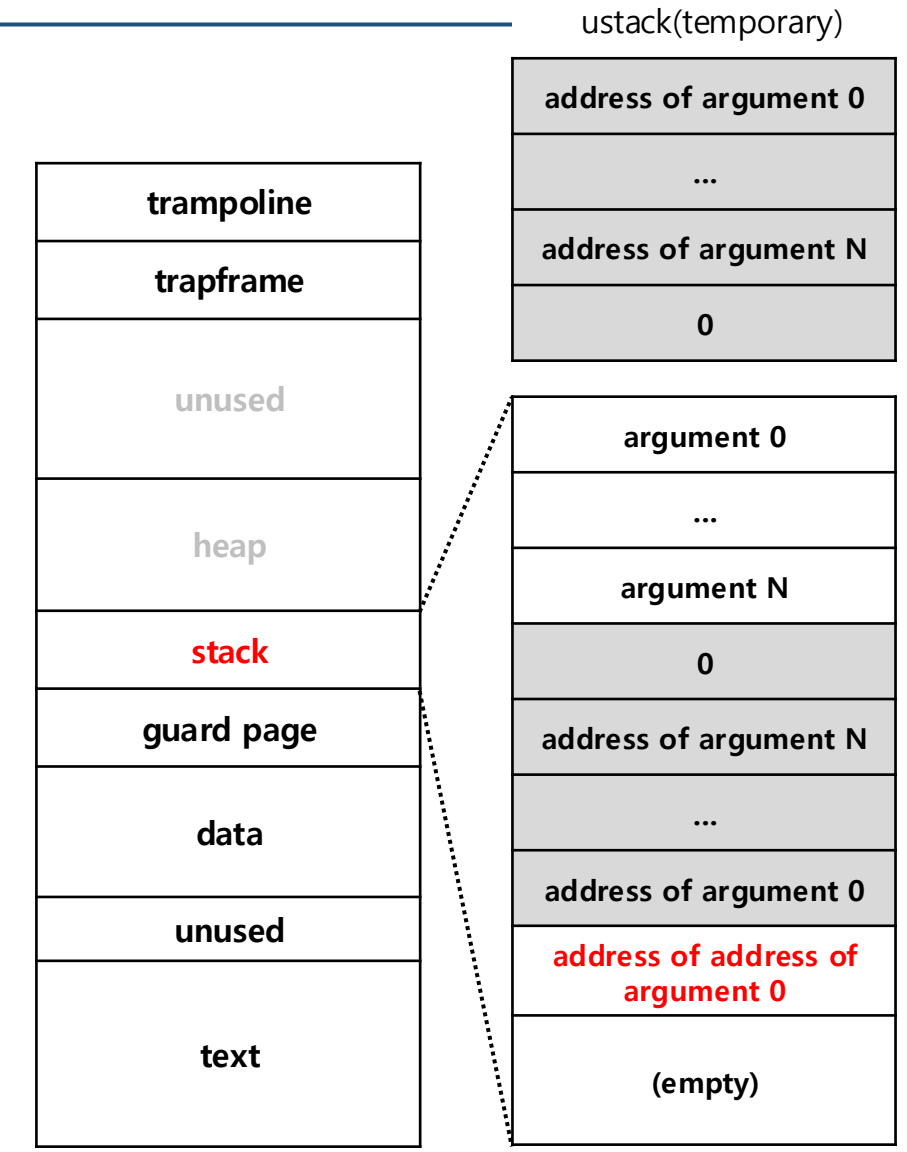
8. Pushes the actual argument strings onto the top of the stack.
9. Stores their addresses in a temporary array (ustack).



exec

```
$ vim kernel/exec.c

 27   kexec(char *path, char **argv)
...
112     sp -= (argc+1) * sizeof(uint64);
113     sp -= sp % 16;
114     if(sp < stackbase)
115       goto bad;
116     if((copyout(pagetable, sp, (char *)ustack, (argc+1)*size ...
117       goto bad;
118
119     // a0 and a1 contain arguments to user main(argc, argv)
120     // argc is returned via the system call return
121     // value, which goes in a0.
122     p->trapframe->a1 = sp;
...
```

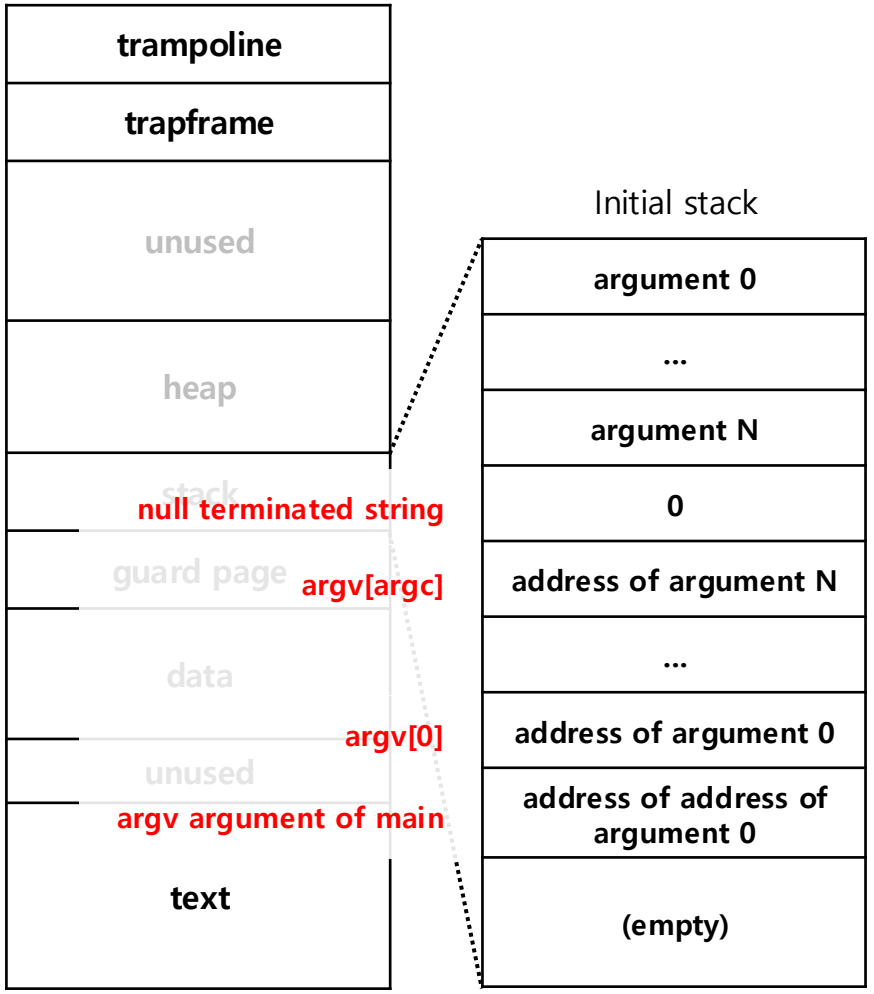
10. Copies the array of argument pointers from temporary `ustack` to the user stack.
11. Sets the `a1` register to `sp` to pass `argv` to the new program.



exec

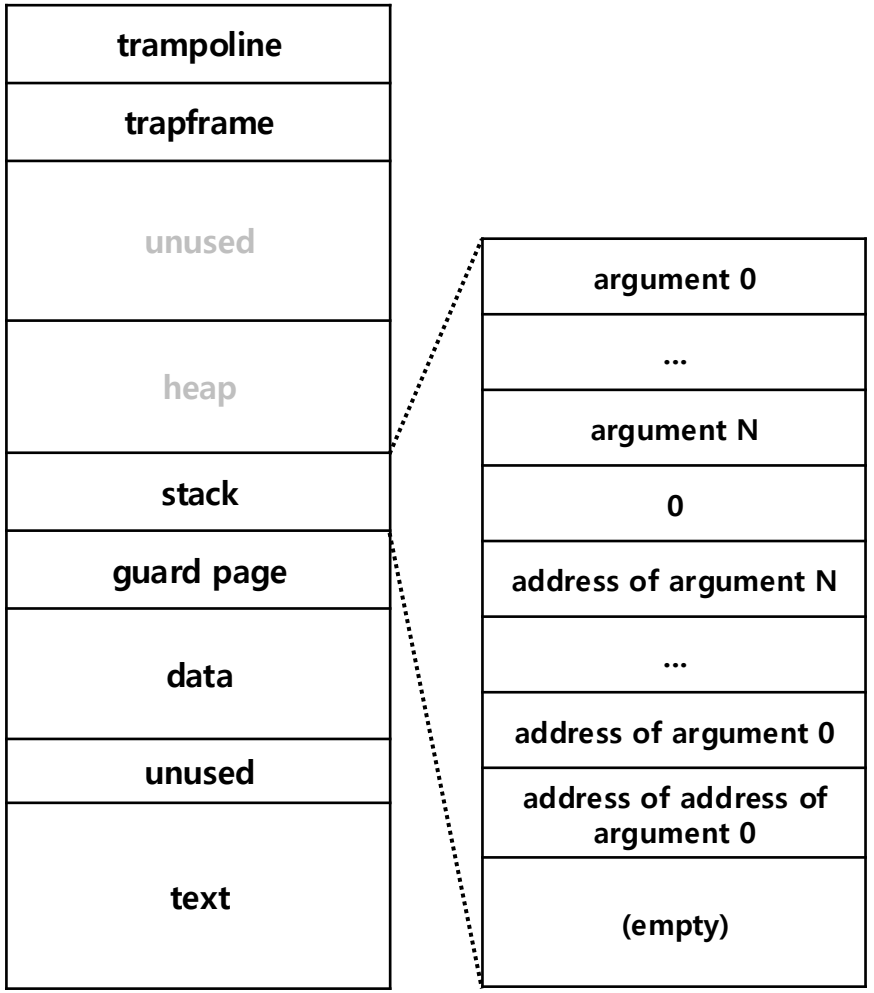
```
$ vim kernel/exec.c

 27   kexec(char *path, char **argv)
...
112     sp -= (argc+1) * sizeof(uint64);
113     sp -= sp % 16;
114     if(sp < stackbase)
115       goto bad;
116     if((copyout(pagetable, sp, (char *)ustack, (argc+1)*size ...
117       goto bad;
118
119     // a0 and a1 contain arguments to user main(argc, argv)
120     // argc is returned via the system call return
121     // value, which goes in a0.
122     p->trapframe->a1 = sp;
...
```

10. Copies the array of argument pointers from temporary `ustack` to the user stack.
11. Sets the `a1` register to `sp` to pass `argv` to the new program.



exec

```
$ vim kernel/exec.c

 27   kexec(char *path, char **argv)
...
130    // Commit to the user image.
131    oldpagetable = p->pagetable;
132    p->pagetable = pagetable;
133    p->sz = sz;
134    p->trapframe->epc = elf.entry; // initial program counter = ...
135    p->trapframe->sp = sp; // initial stack pointer
136    proc_freepagetable(oldpagetable, oldsz);
137
138    return argc; // this ends up in a0, the first argument to
...                    main(argc, argv)

$ vim user/any_main.c

  1   int
  2   main(int argc, char *argv[])
  3   {
```



12. Commits to the new page table and free the old memory.
13. Sets the program counter to the entry point and returns argc.

wait

- The `wait()` system call allows a parent process to wait for its child to terminate.
- It retrieves the child's **exit status** and cleans up **zombie** processes by releasing their resources.
- If no children have exited, the parent process is put to sleep until one does.

wait

```
$ vim kernel/proc.c

370   int
371   kwait(uint64 addr)
372   {
...
379     for(;;){
...
382       for(pp = proc; pp < &proc[NPROC]; pp++){
383         if(pp->parent == p){
...
388           if(pp->state == ZOMBIE){
389             // Found one.
390             pid = pp->pid;
391             if(addr != 0 && copyout(p->pagetable...
...
396             freeproc(pp);
...
400             return pid;
401           }
...
```

1. Continuously loops to check if any child process has exited.
   - It scans the entire process table to find child processes of the current process.
2. If a child process in the `ZOMBIE` state is found, it retrieves the **exit status**, frees the child's resources, and returns the child's process ID

wait

```
$ vim kernel/proc.c

370   int
371   kwait(uint64 addr)
372   {
...
379     for(;;){
380       // Scan through table looking for exited children.
381       havekids = 0;
382       for (pp = proc; pp < &proc[NPROC]; pp++){
...
404       }
405
406       // No point waiting if we don't have any children.
407       if(!havekids || killed(p)){
408         release(&wait_lock);
409         return -1;
410       }
411
412       // Wait for a child to exit.
413       sleep(p, &wait_lock); //DOC: wait-sleep
414     }
415   }
```

3. If the process has no children or the parent itself is killed, it returns -1 immediately.
4. If no child has exited yet, the parent **sleeps until a child process calls `exit()`**.

exit

- The `exit()` system call terminates the current process.
- The terminated process enters a “zombie state”, remaining in the system until its parent process collects it using wait().
- Exit status can be passed as an argument to the parent process.

exit

```
$ vim kernel/proc.c

326   void
327   kexit(int status)
328   {
...
331     if(p == initproc)
332       panic("init exiting");
333
334     // Close all open files.
335     for(int fd = 0; fd < NOFILE; fd++){
336       if(p->ofile[fd]){
337         struct file *f = p->ofile[fd];
338         fileclose(f);
339         p->ofile[fd] = 0;
340       }
341     }
342
343     begin_op();
344     iput(p->cwd);
345     end_op();
346     p->cwd = 0;
...
```

1. Checks if the current process is an init process.
   - The root process `initproc` must not be allowed to exit.
2. Closes all open files and removes them from the file table.
3. Releases the process's current working directory by dropping a reference to an inode.

exit

```
$ vim kernel/proc.c

327   kexit(int status)
...
348     acquire(&wait_lock);
349
350     // Give any children to init.
351     reparent(p);
352
353     // Parent might be sleeping in wait();
354     wakeup(p->parent);
355
356     acquire(&p->lock);
357
358     p->xstate = status;
359     p->state = ZOMBIE;
360
361     release(&wait_lock);
362
363     // Jump into the scheduler, never to return.
364     sched();
365     panic("zombie exit");
366   }
```

4. Reassigns child processes to the `initproc` to prevent them from becoming orphans.

```
$ vim kernel/proc.c

310  void
311  reparent(struct proc *p)
312  {
313    struct proc *pp;
314
315    for(pp = proc; pp < &proc[NPROC]; pp++){
316      if(pp->parent == p){
317        pp->parent = initproc;
318        wakeup(initproc);
319      }
320    }
321  }
```

exit

```
$ vim kernel/proc.c

327   kexit(int status)
...
348     acquire(&wait_lock);
349
350     // Give any children to init.
351     reparent(p);
352
353     // Parent might be sleeping in wait();
354     wakeup(p->parent);
355
356     acquire(&p->lock);
357
358     p->xstate = status;
359     p->state = ZOMBIE;
360
361     release(&wait_lock);
362
363     // Jump into the scheduler, never to return.
364     sched();
365     panic("zombie exit");
366   }
```

4. Reassigns child processes to the `initproc` to prevent them from becoming orphans.
5. If the parent is sleeping in `wait()`, it is woken up to allow cleanup.
6. Sets its exit status value, and its state to `ZOMBIE`.
7. After these steps, the exiting process calls `sched()` to jump into the scheduler, never to return.

kill

- The `kill()` system call sends a termination request to the process with the specified PID.
- It sets the `killed` flag, and if the process is sleeping, it changes its state to `RUNNABLE` to wake it up.
- The process will actually terminate when it returns to user space.

kill

```
$ vim kernel/proc.c

592   int
593   kkill(int pid)
594   {
595     struct proc *p;
596
597     for(p = proc; p < &proc[NPROC]; p++){
598       acquire(&p->lock);
599       if(p->pid == pid){
600         p->killed = 1;
601         if(p->state == SLEEPING){
602           // Wake process from sleep().
603           p->state = RUNNABLE;
604         }
605         release(&p->lock);
606         return 0;
607       }
608       release(&p->lock);
609     }
610     return -1;
611   }
```

1. Scans the entire process table and finds the process with the matching PID.
2. If found, it sets the process's killed field to 1 and changes its state to `RUNNABLE` to wake it up if it is sleeping.

When does the killed process exit?

```
$ vim kernel/trap.c

 37   uint64
 38   usertrap(void)
 39   {
...
 80     if(killed(p))
 81       exit(-1);
...
 93     return satp;
 94   }
```

```
$ vim kernel/proc.c

621   int
622   killed(struct proc *p)
623   {
...
626     acquire(&p->lock);
627     k = p->killed;
628     release(&p->lock);
629     return k;
630   }
```

- When a trap occurs in user space, the trap handler checks the current process's `killed` field.
  - If so, it exits the current process with a -1 status.

Why we use killed flag?

To prevent the kernel state from being corrupted !!

Recap

- System calls related to process execution
  - `fork(void)`
  - `exec(const char *, char **)`
  - `wait(int *)`
  - `exit(int)`
  - `kill(int)`

Thank you